BECKHOFF

Special Projects Team

TwinCAT HMI Technical Guide

v1.2

July 10, 2026

Contents

Default Styling 4
CSS Files 4
Project Primary Highlight Color 4
Custom Navigation Styling 4
General Styles 6
UI Component Specific Styles 6
Theme Files 9
Control Classes 9
Control Types 11
Themed Resource Symbols 12
Grid and Flexbox Setup 14
PackML Control Setup 15
SPT Framework v3/4 15
SPT Core 16
Editing Available PackML Modes 17
SPT Core Manual Page Auto-Generation 18
Porting PackML and Panel Controls to an Existing Project 21
Updating Server Extension Targets 22
System Page 24
Project Renaming 24
Option 1 - Basic Rename 25
Option 2 - Create a Local Visual Studio Template 27
Font Usage Guidelines 30
Adding Icons 31
Adding Additional Folders 31
Project Handoff 32

Default Styling

This section details additions made to CSS and Theme files for the purpose of creating a consistent and visually appealing UI. If differences exist between light and dark themes, they will be noted, otherwise it can be assumed the style applies to both. In addition, Themed Resource Symbols are discussed.

CSS Files

Project Primary Highlight Color

```
:root {
    --tchmi-highlight-color-1: #6610F2;
}
```

Within TcHMI, CSS variables are utilized so that style values may be reused in many places. Among these variables is a highlight color, which we have changed for the purpose of using a more identifiable color and to illustrate where and how to change this as easily as possible to suit a customer's needs. Change the color value here to update the primary color in the project. This will commonly be done and should be a company's or product's main color. Please note that the highlight color Control Class and Themed Resource Symbol should be updated as well.

While the above takes care of a majority of highlight colors in HMI controls, some need to be targeted a little more specifically. The below rule handles these cases.

```
.TcHmi_Controls_Helpers_Popup-header-container {
    background: var(--tchmi-highlight-color-1);
}
```

Custom Navigation Styling

Light Theme -

```
/* icon inactive */
.TcHmi_Controls_BaseTemplate_TcHmiAccordionNavigation i, .tchmi-class-icon-dark img, .tchmi-class-icon-dark .TcHmi_Controls_Beckhoff_TcHmiButton-template, .tchmi-class-icon-dark[data-tchmi-background-image] {
    filter: brightness(0.2);
}

/* icon active */
```

```
.TcHmi_Controls_BaseTemplate_TcHmiAccordionNavigation-Template > tchmi-accordion > tchmi-
accordion-item[active] > tchmi-accordion-header i {
    filter: brightness(1);
}

/* navigation button text */
tchmi-accordion > tchmi-accordion-item > tchmi-accordion-header {
    color: #000000;
}

/* selected button color */
.TcHmi_Controls_BaseTemplate_TcHmiAccordionNavigation {
    --tchmi-highlight-color-1: #b9b9b9;
}
```

Dark Theme –

```
/* icon inactive */
.TcHmi_Controls_BaseTemplate_TcHmiAccordionNavigation i {
    filter: brightness(1);
}

/* icon active */
.TcHmi_Controls_BaseTemplate_TcHmiAccordionNavigation-Template > tchmi-accordion > tchmi-
accordion-item[active] > tchmi-accordion-header i {
    filter: brightness(0.2);
}


/* selected navigation button text */
.TcHmi_Controls_BaseTemplate_TcHmiAccordionNavigation-Template > tchmi-accordion > tchmi-
accordion-item[active] > tchmi-accordion-header {
    color: #000000;
}

/* selected button color */
.TcHmi_Controls_BaseTemplate_TcHmiAccordionNavigation {
    --tchmi-highlight-color-1: #b9b9b9;
}
```

Here the rules with the ‘filter: brightness(x)’ style targets icons. The purpose of this is to eliminate the need to maintain and account for multiple icon files when switching themes or button/icon states (ie pressed, active, inactive, etc). Note that this also applies to image controls and buttons with icons using the icon-dark control class applied. The icon-dark control class is only needed for light theme as icon default colors align with dark theme.

Note that icon color filtering is inverted between light and dark themes. This ensures a light colored icon on a dark background and a dark colored icon on a light background.

When a navigation button is selected, the defauolt behavior is for the highlight color CSS variable to be used. In the case of navigation, we do not want to use this color, so for the scope of the accordion navigation control, we change the variable's value to grey.

General Styles

Global styling to apply universal rounded corners and font size throughout the project. The font size unit cqh (container query height) is utilized as a means of automatically scaling height with the height of the viewport/screen. This may need to altered as needed, for example, if an HMI is to be viewed in portrait, cqw (container query width) would be more appropriate. Also included is a lower bounds of 10px to keep text readable at smaller screen sizes.

Note that setting a font size directly on a control will override this rule, which can be beneficial in cases of smaller control within an HMI. In these cases, it is recommended to use '%' font size unit so the text will still scale.

```
/* round corners */
.tchmi-control, tchmi-accordion-header, .tchmi-in-topmostlayer {
    border-radius: 0.45rem;
}

/* global font size */
.tchmi-box, .notyf__message {
    font-size: max(10px, 1.8cqh);
}
```

UI Component Specific Styles

Style Guidelines

Component specific styles to ensure conformance with style guidelines.

```
/* popup header text */
.TcHmi_Controls_Helpers_Popup-header-container a, .TcHmi_Controls_Helpers_Popup-header-
container h1 {
    filter: brightness(100)
}

/* popup and keyboard background color (see theme specific callouts below) */


/* apply border radius to multi-part controls eg controls with a label */
.TcHmi_Controls_Beckhoff_TcHmiCheckbox-template {
    border-radius: inherit;
}

/* style images like buttons */
```

```
.tchmi-class-icon-button {
    background: var(--tchmi-button-background);
    color: var(--tchmi-button-text-color);
    border: var(--tchmi-button-border);
    box-shadow: var(--tchmi-button-shadow);
}

.tchmi-class-icon-button:active {
    background: var(--tchmi-button-background-pressed);
    color: var(--tchmi-button-text-color-pressed);
    border: var(--tchmi-button-border-pressed);
    box-shadow: var(--tchmi-button-shadow-pressed);
}

/* simple grid gap control class */
.tchmi-class-grid-gap .TcHmi_Controls_System_TcHmiGrid-grid {
    gap: 10px;
}

.tchmi-class-grid-gap > div > div > .tchmi-grid-gridrow {
    column-gap: 10px;
}

.TcHmi_Controls_System_TcHmiFlexContainer.tchmi-class-grouping-container,
.TcHmi_Controls_System_TcHmiGridContainer.tchmi-class-grouping-container, .tchmi-class-grid-
padding{
    padding: 10px;
}
```

Light Theme –

```
/* popup and keyboard background color */
.TcHmi_Controls_Helpers_Popup-content, .TcHmi_Controls_Helpers_Popup-footer,
.TcHmi_Controls_Beckhoff_TcHmiKeyboard {
    background-color: #E3E6E9;
}
```

Dark Theme –

```
/* popup and keyboard background color */
.TcHmi_Controls_Helpers_Popup-content, .TcHmi_Controls_Helpers_Popup-footer,
.TcHmi_Controls_Beckhoff_TcHmiKeyboard {
    background-color: #2A2929;
}
```

Light Theme Only –

```
/* gauge colors */
.TcHmi_Controls_Beckhoff_TcHmiTachometer, .TcHmi_Controls_Beckhoff_TcHmiLinearGauge,
.TcHmi_Controls_Beckhoff_TcHmiRadialGauge, .tchmi-linear-gauge {
    --tchmi-tick-stroke: black;
```

```
    --tchmi-label-fill: black;
    --tchmi-unit-color: black;
    --tchmi-label-color: black;
    --tchmi-description-color: black;
    --tchmi-tick-background: black;
}
```

PackML Flyout Functionality

Component specific styles for animation and other functionality of PackML flyout

```
/* used to allow for clicks to pass through specific parts of a control unit */
.tchmi-class-no-interaction {
    touch-action: none;
    pointer-events: none;
}

.tchmi-class-interaction {
    touch-action: initial;
    pointer-events: initial;
}

/* rules and animation for packml flyout */
#desktop-grid-machinecontrol {
    --pmlcontrol-column-width: 160px;
}

.tchmi-class-slide-in {
    animation: 250ms 1 forwards slide-in;
}

@keyframes slide-in {
    from {
        right: 0px;
    }

    to {
        right: calc(-1 * var(--pmlcontrol-column-width));
    }
}

.tchmi-class-slide-out {
    animation: 250ms 1 forwards slide-out;
}

@keyframes slide-out {
    from {
        right: calc(-1 * var(--pmlcontrol-column-width));
    }

    to {
        right: 0px;
    }
}
```

Theme Files

Control Classes

background-[color] –

These are meant to echo the semantic colors defined in the style guide, as well as the primary highlight color. Included here so that they can be easily applied to the background color property of buttons, inputs, etc. through project development.

highlight – default #6610F2 (Should be changed if highlight color in css is changed)

green - #51D777

orange - #FBA712

red - #EF4949

blue - #175DB8

grey - #B9B9B9

grouping-container –

To be applied to containers, grids, or flex boxes which are used to groups related controls together.

Border Style: Solid

Border Width: 1px

Light Theme –

Background Color: rgba(255, 255, 255, 128)

Border Color: rgba(220, 220, 220, 255)

Dark Theme –

Background Color: rgba(225, 255, 255, 12)

Border Color: rgba(70, 70, 70, 255)

icon-button –

For CSS styling. Used to style image controls so they look like buttons.

icon-dark –

For CSS styling, light theme only. Filters icon color to be dark when placed on a light background.

main-header –

Gives header a bottom border with 1px width and coloring of background and border to differentiate header from different sections.

Light Theme:

Background Color: rgba(246, 247, 248, 128)

Border Color: rgba(220, 220, 220, 255)

Dark Theme:

Background Color: rgba(37, 36, 36, 255)

Border Color: rgba(70, 70, 70, 255)

main-nav –

Gives nav bar a right border with 1px width and coloring of background and border to differentiate navigation from different sections.

Light Theme:

Background Color: rgba(246, 247, 248, 128)

Border Color: rgba(220, 220, 220, 255)

Dark Theme:

Background Color: rgba(37, 36, 36, 255)

Border Color: rgba(70, 70, 70, 255)

text-centered –

Centers text vertical and horizontal alignment.

grid-gap –

For CSS styling. To be applied to grids, in tandem with grid-padding, for easily setting up proper gaps between rows, columns, and controls.

grid-padding –

For CSS styling. To be applied to grids, in tandem with grid-gap, for easily setting up proper padding between rows, columns, and controls.

no-interaction –

For CSS styling. Applied to the PackML Machine Control grid to allow touches and clicks to pass through blank portions of the grid. Applying to grid also applies this to the controls within. See interaction class below.

interaction –

For CSS styling. Applied to Machine Control Show/Hide Button and PackML Control User Control to set touch and click events back to original state, allowing for interaction with these controls.

slide-in –

For CSS styling. Animation to slide PackML Control in, hiding it from view.

slide-out –

For CSS styling. Animation to slide PackML Control out, bringing it back into view.

Control Types

Combobox/Localizaiton Select/Theme Select –

Data Height: 40px

Max List Height: 200px

Input/Textbox/Numeric Input/Labeled Numeric Input/ Labeled Input –

Auto Select Text: True

Control –

Flex Item Layout: Flex 1 1 auto

DataDisplayTile –

BorderColor: rgb(220, 220, 220)(Light)/rgb(80, 80, 80)(Dark)

BorderStyle: Solid

BorderWidth: 1px

Flex Container –

FlexLayout: ColumnGap/RowGap 10px

Grid Container –

GridLayout: ColumnGap/RowGap 10px

Themed Resource Symbols

Symbols that can be used for data bindings to control color properties. Can be changed in the TwinCAT HMI Configuration window.

ChartAxisColors –

Used for Charts to color labels and lines based on theme.

Light Theme: #000000

Dark Theme: #FFFFFF

Default[Colors] –

Used to echo default semantic colors defined in the style guide. Included to ease setting of colors where control classes aren't as useful.

DefaultGreen –

#51D777

DefaultOrange –

#FBA712

DefaultRed –

#EF4949

DefaultBlue –

#175DB8

DefaultGrey –

#B9B9B9

HighlightColor –

#6610F2 (Should be changed if highlight color in css is changed)

Grid and Flexbox Setup

The GridContainer and FlexContainer controls are wrappers/analogs to true CSS Grids and Flexbox. They provide means of easily configuring a page layout with clean alignment and spacing. In general, Grid should be used for layouts that are in 2 dimensions, while flex is used for 1 dimension layouts.

When should grid and flex containers be used rather than the standard grid or container? For more complex layout and groups of multiple controls. They may be overkill for simple layouts and a "group" of one control. Overall, these CSS controls are a lot more light weight and performant than the standard TcHmiGrid.

Setup –

1. Start with a grid and define rows, columns, and gaps (always 10px).
2. Add grouping containers to rows/columns as desired. Use another grid here if there will be multiple rows of controls, use Flex if controls will be on a single row
3. Give grouping containers appropriate grid column/row definitions and add grouping-container control class
4. Define grid/flex layout appropriately. Note that row gap and column gap should always be 10px. These are set as project defaults, but if you change anything on a specific control instance, make sure to set them to 10px otherwise the default may be overwritten. Justify-x and Align-x properties may need to be set if controls are not set to take up an entire flex/grid cell/area. See Documentation>css-flexbox/grid-quick-reference or search MDN for explanation on how this works.
5. When adding controls to a Flex box, set Flex Item Layout property (have to expand Layout's hidden section). If all controls in the flex box should be the same size, set Flex to 1 1 auto. Else define as makes sense. See Documentation>css-flexbox -quick-reference or search MDN for explanation on how this works. NOTE for the auto to work, a width property must be set on the control, it can be somewhat arbitrary, but still reasonable.
6. Make sure the width and height are set and Left and Top are 0. In general, height of 100% is good here. (HOWEVER, if you changed the FlexContainer's FlexLayout.FlexDirection to Column, you would want to set height to an arbitrary but reasonable value and width to 100%). For grids, Width and Height can both be 100%.
7. Some modification to sizing, positioning, and alignment can be made, but in general, make sure a clean and consistent look is achieved.

PackML Control Setup

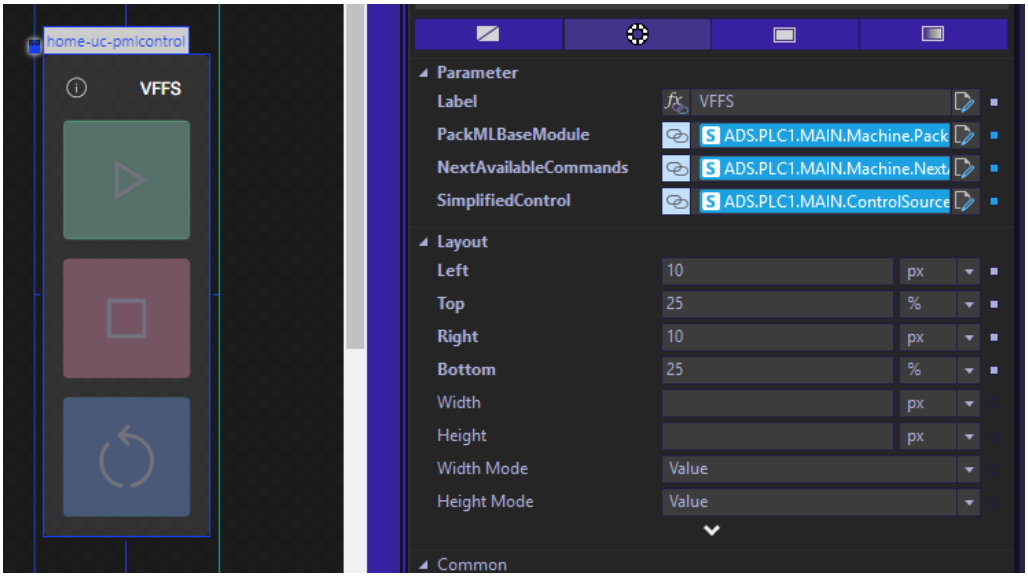
The PackML Control included in the Template project may require some configuration. This setup can vary based on SPT Framework version, which is outlined below. The PackMLControl User Control is included on the Home page for the machine level by default, but can be used on other pages and for specific Equipment Modules/Components. The PackML Selection user control is also available, but is not to be implemented by the HMI developer, as it is utilized in a popup triggered by the PackMLControl UC.

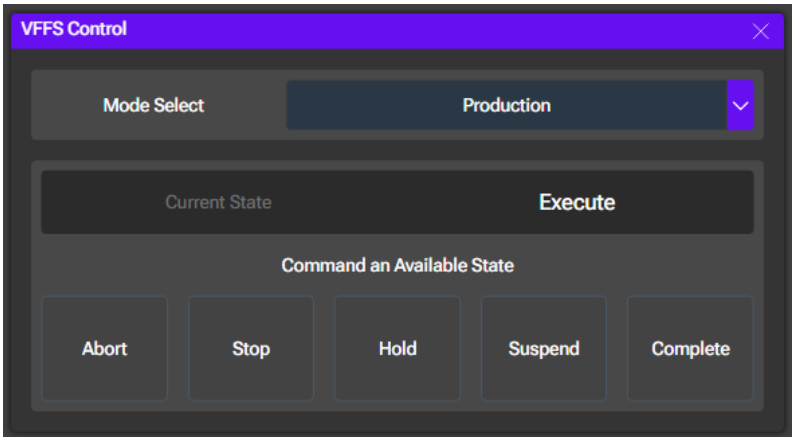
SPT Framework v3/4

Within the SPT Framework v3 and v4, configuration is done by giving a Label to be used on the control and it associated popup's header. This can be the Machine or EM's name.

Finally, the PackMLBaseModule_HMI, NextAvailableCommands, and ControlSourceSimplified symbols from the library are bound to the User Control parameters.

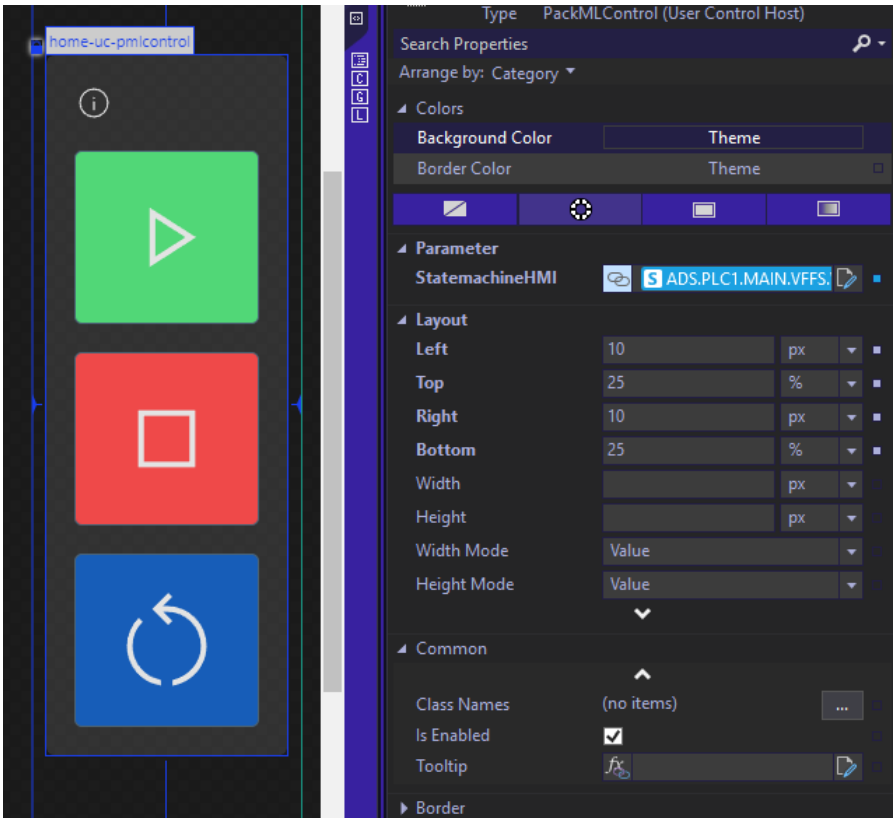
The control takes care of the rest and displays the current Mode and State, as well as allows changing of mode and commanding of any available states.

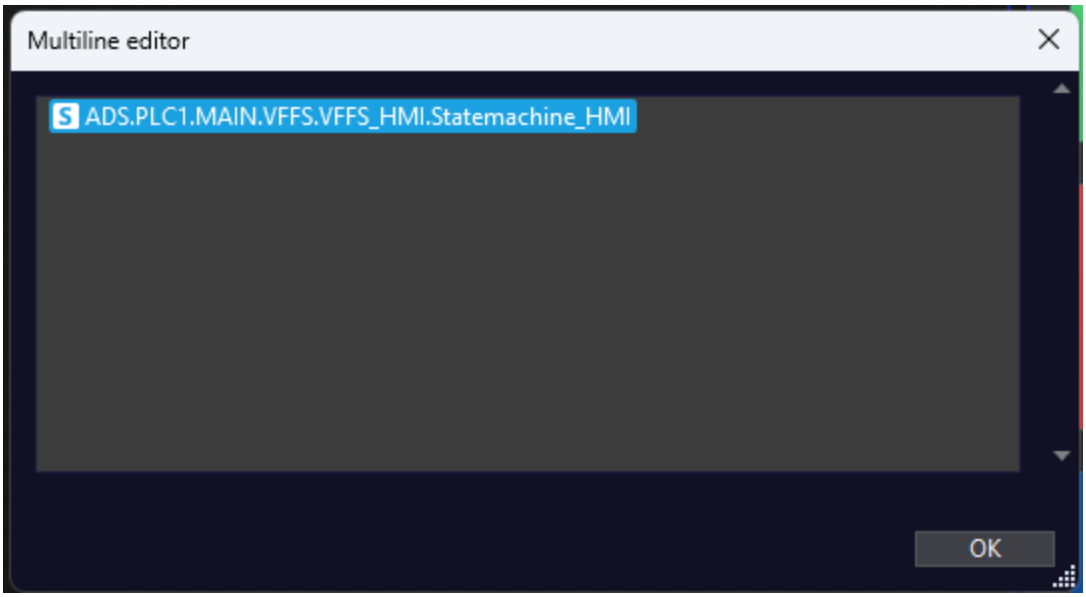




SPT Core

With SPT Core, setup is simplified even further. Only the Machine's or EM's StateMachine_HMI Structure needs to be bound to the StatemachineHMI Parameter. Just like before, the control and library take care of the rest.

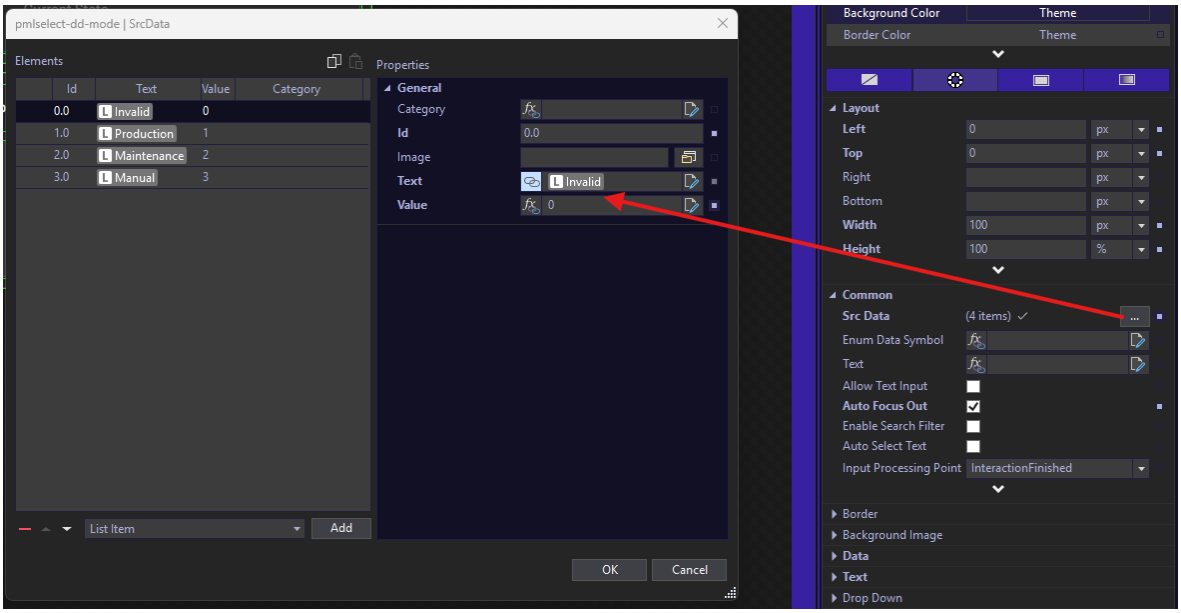




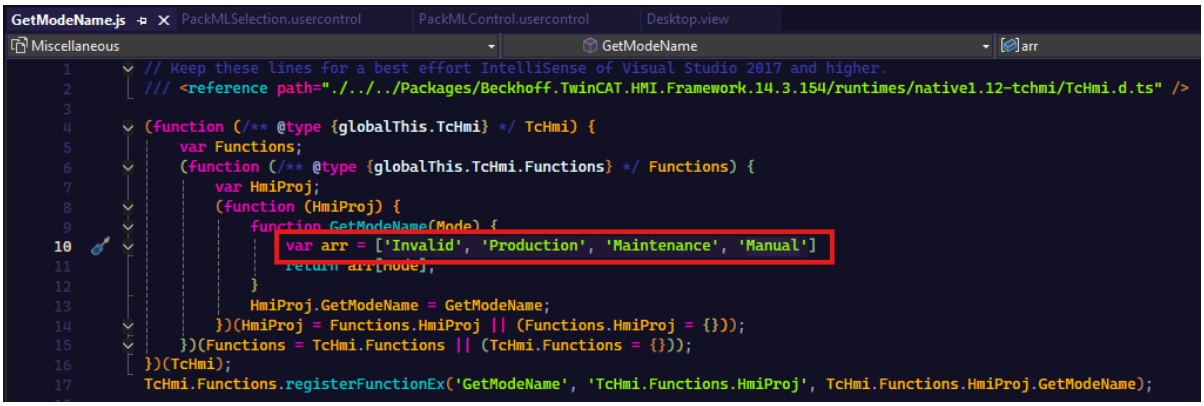
Editing Available PackML Modes

By default, the included PackML Modes in the template are Invalid, Production, Maintenance, Manual. If it is necessary to add or remove modes, two places must be updated. These updates are manual due to Mode coming in as purely an Integer from the PLC.

The dropdown in the popup relies on the SrcData property. Add, remove, or edit list items here as needed and ensure the proper numeric value is associated with each mode.



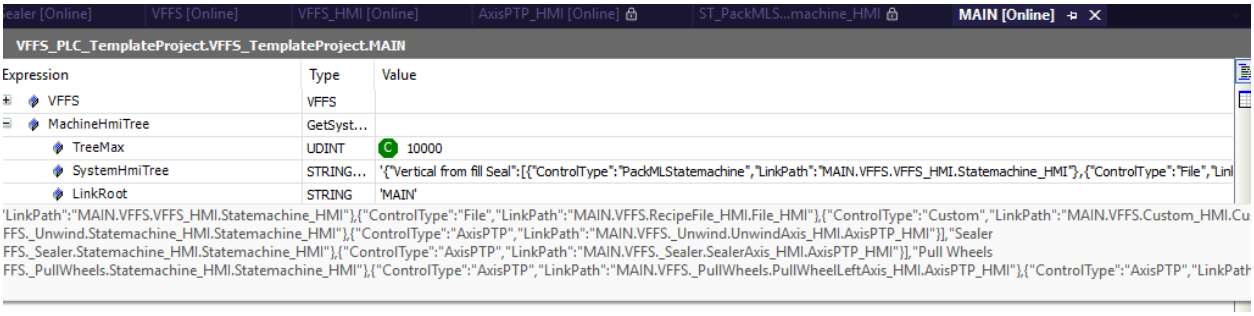
Additionally, there is a small script that handles converting the numeric value to the name for display purposes. In GetModeName.js, update the array to reflect the included mode names. Note that the index of the name must match the numeric value associated with it.



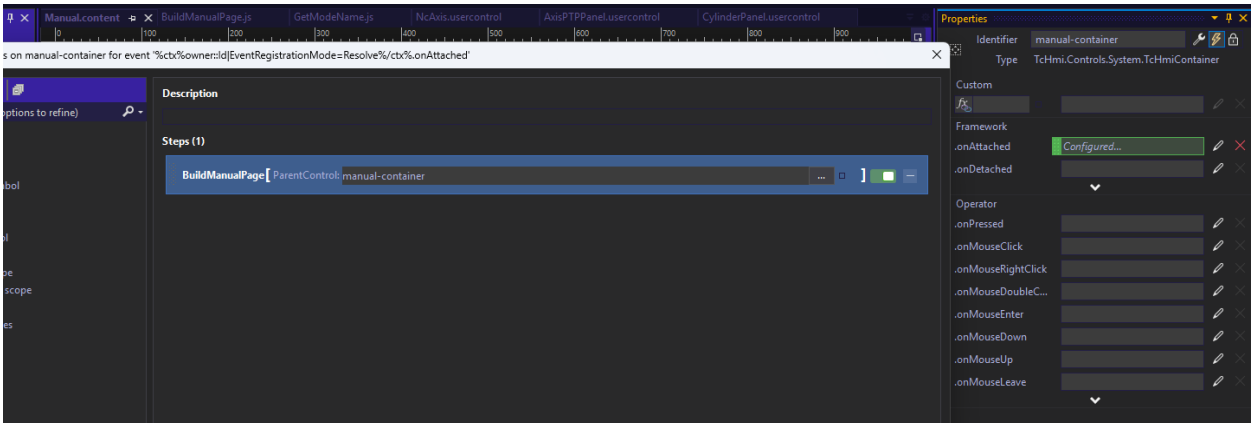
SPT Core Manual Page Auto-Generation

Included with Core is the ability to autogenerate a Manual page with controls for the machine and each equipment module and component.

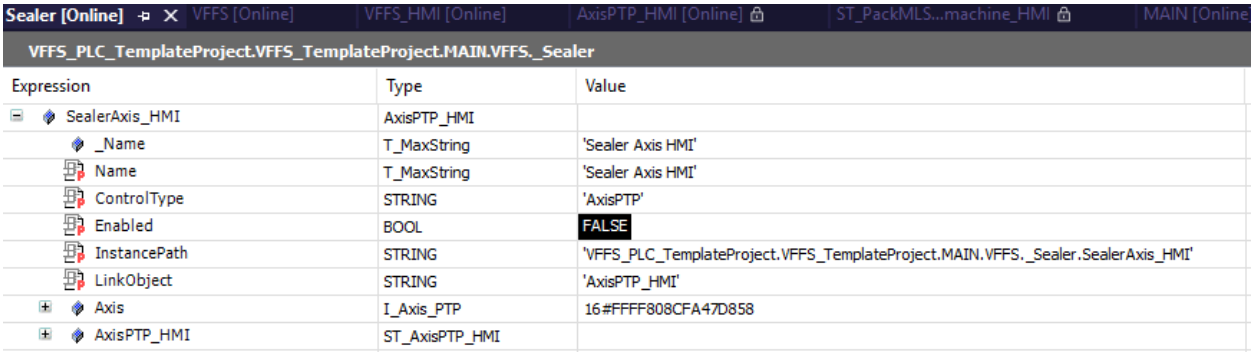
After standard creation of the machine, EMs, and components in the PLC, the library creates a JSON string which is sent to the HMI. This JSON string, importantly, contains the Control Type and a Link Path of the HMI Symbol.

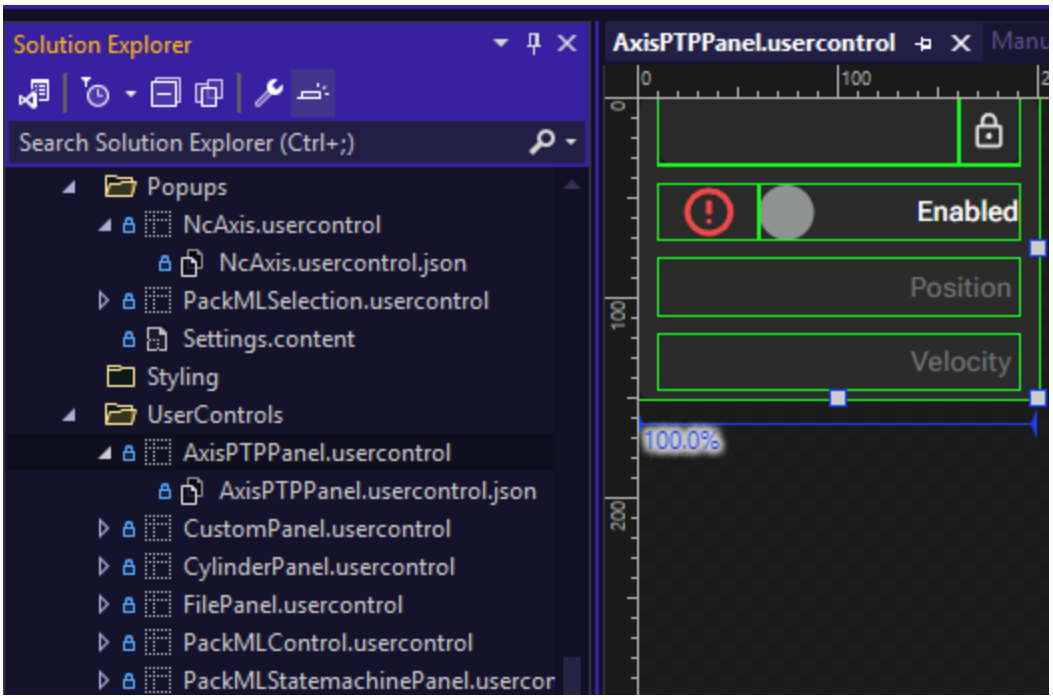


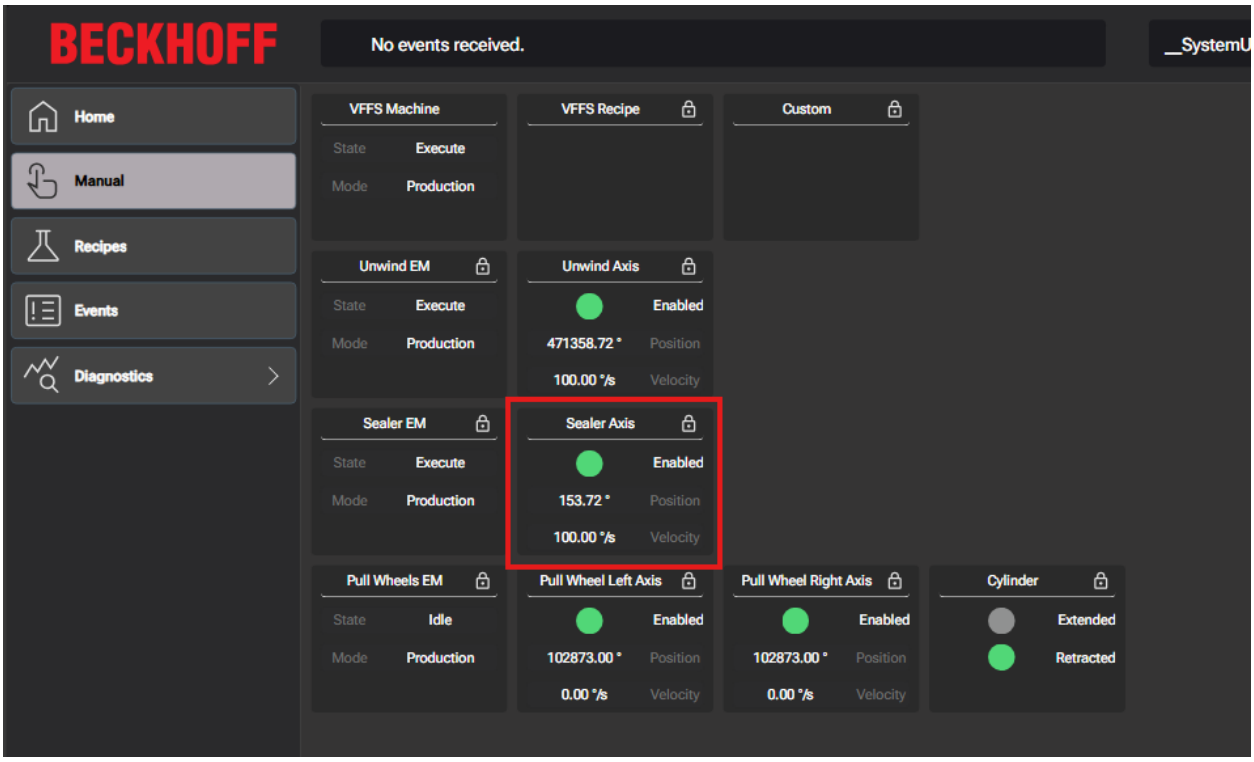
The Manual Page has a container which runs the BuildManualPage function with a reference to the container passed as a parameter when it is attached.



In order to take advantage of this function, the HMI developer must create the appropriate User Controls which will correspond to the PLC defined Control Types. **The User Control name must match the Control Type plus 'Panel'.** For example, if there is an AxisPTP Control type, the associated UC must be called AxisPTPPanel.usercontrol




Solution Explorer
Search Solution Explorer (Ctrl+;)
Popups
NcAxis.usercontrol
NcAxis.usercontrol.json
PackMLSelection.usercontrol
Settings.content
Styling
UserControls
AxisPTPPanel.usercontrol
AxisPTPPanel.usercontrol.json
CustomPanel.usercontrol
CylinderPanel.usercontrol
FilePanel.usercontrol
PackMLControl.usercontrol
PackMLStatemachinePanel.usercor
AxisPTPPanel.usercontrol
Enabled
Position
Velocity
100.0%


BECKHOFF
No events received.
Home
Manual
Recipes
Events
Diagnostics
VFFS Machine
State Execute
Mode Production
VFFS Recipe
Custom
Unwind EM
State Execute
Mode Production
Unwind Axis
Enabled
471358.72 °
Position
100.00 °/s
Velocity
Sealer EM
State Execute
Mode Production
Sealer Axis
Enabled
153.72 °
Position
100.00 °/s
Velocity
Pull Wheels EM
State Idle
Mode Production
Pull Wheel Left Axis
Enabled
102873.00 °
Position
0.00 °/s
Velocity
Pull Wheel Right Axis
Enabled
102873.00 °
Position
0.00 °/s
Velocity
Cylinder
Extended
Retracted

Porting PackML and Panel Controls to an Existing Project

The PackML and Panel Controls can be brought into an existing project. Follow the steps below closely to do this. This example is specific to the Pack ML Control, but the process is similar for Panels.

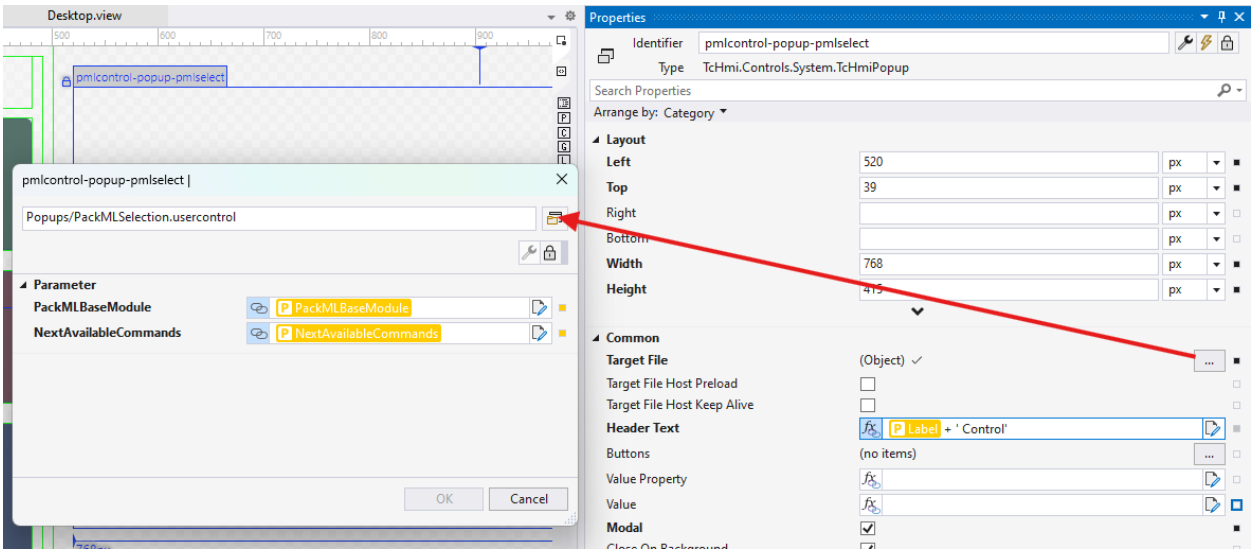
First, simply right-click and select Add>Existing Item in the project's UserControls folder and find and select the PackMLControl user control and again in the project's Popups folder and find and select the PackMLSelection user control.

Once the User Controls are in your project, you can add the PackMLControl UC to any page from the toolbox.

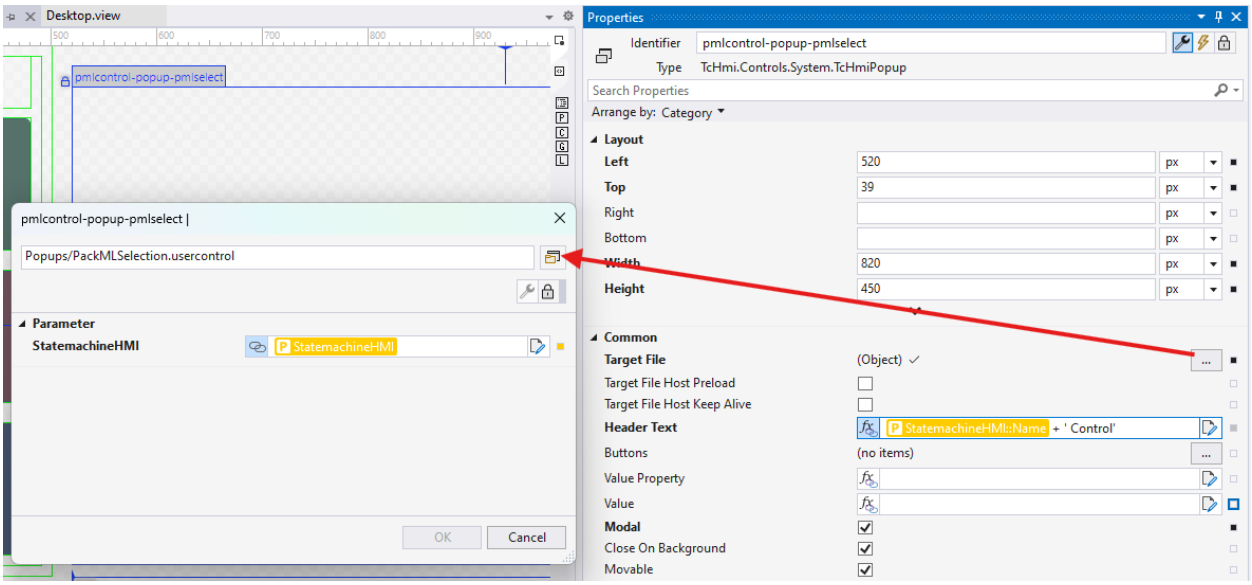
If your project does not follow the same folder structure, be sure to update the path to the PackMLSelection control from within the PackMLControl.usercontrol file.

Select the popup with id pmlcontrol-popup-pmlselect and update its Target File property. If the Parameters passed into PackMLSelection have been reset, rebind them with the PackMLControl parameters exactly as seen below. Noting that these also vary between SPT v3/4 and Core.

v3/4 –

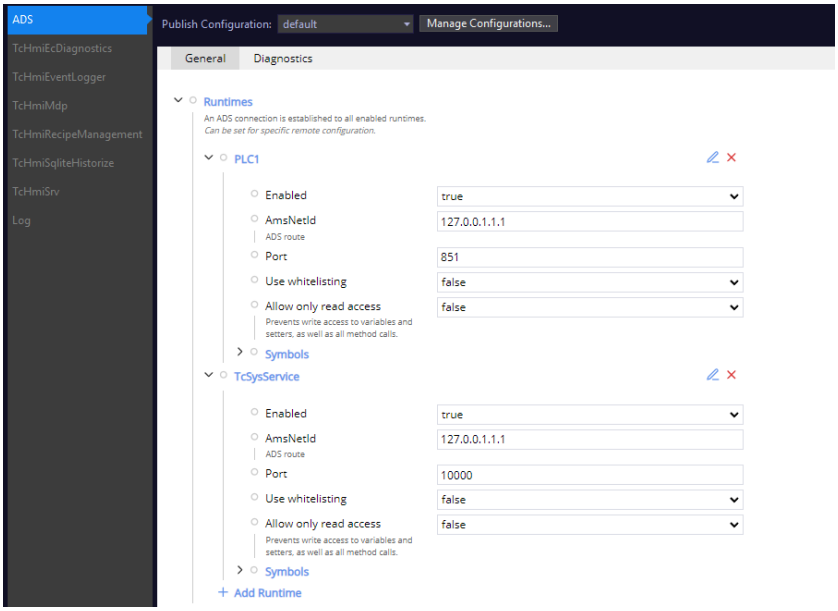


Core –

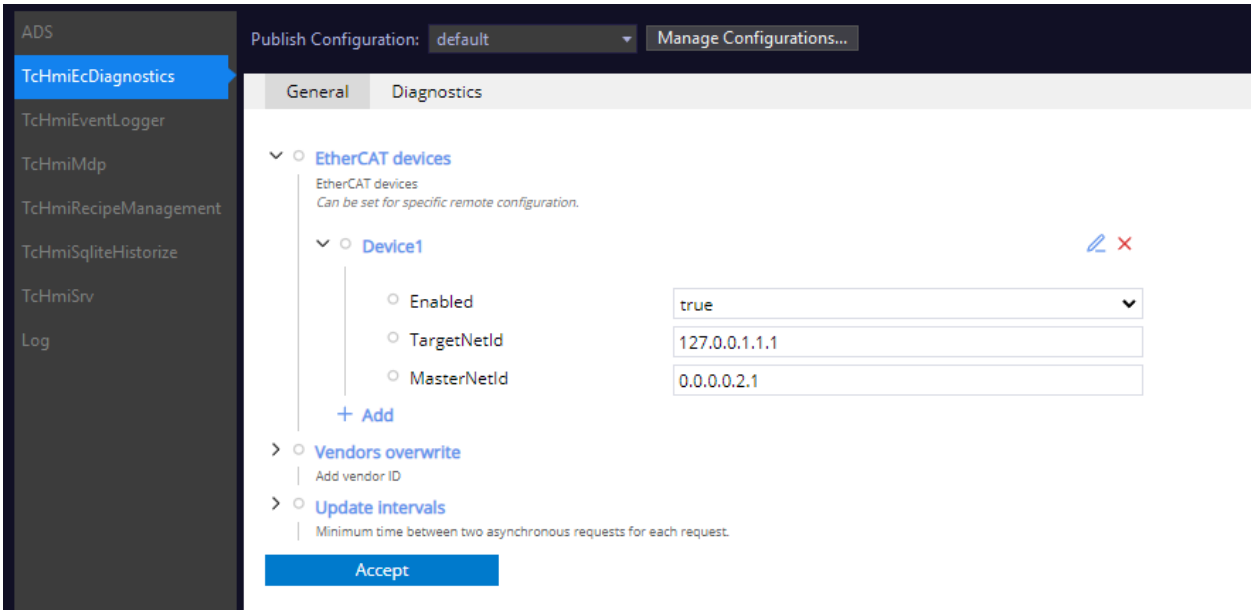


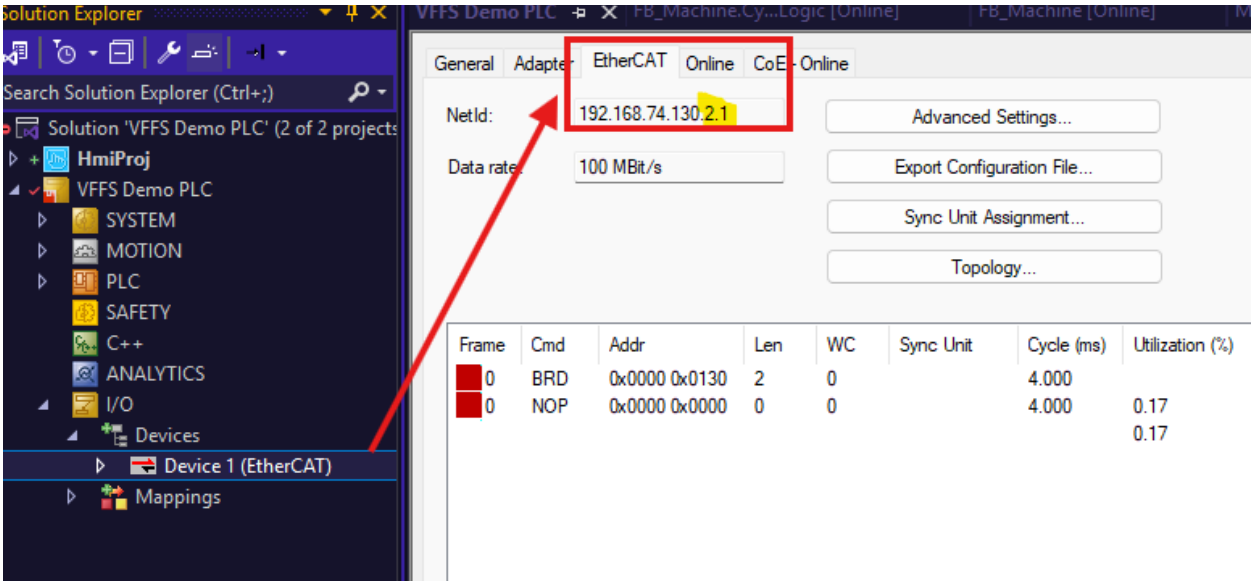
Updating Server Extension Targets

For each new project, it is advisable to update the targets of the HMI Server Extensions when you are connected to a remote PC. At the very least, this should be done for the default publish configuration, as this is how the engineering environment and live view know where to look. This should be done for ADS (both PLC1 and TcSysService), TcHmiEcDiagnostics, TcHmiEventLogger, and TcHmiMdp.



Take special care for the EtherCAT Diagnostics extension for both default and remote. If the Master Net ID varies from the default x.x.x.x.2.1, this change needs to be made to ensure proper functionality of the control.



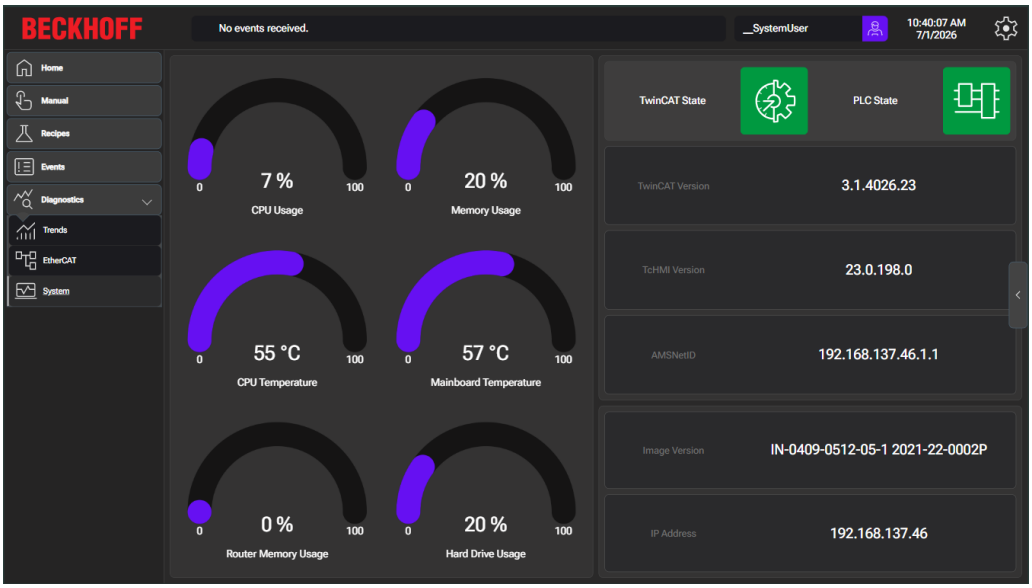


System Page

The system page largely relies on the MDP Extension. It is important to understand that this will only function when connected to a Beckhoff IPC. With the exception of a few controls, it will not work locally.

The default bindings on the controls are generic and should work out of the box, however some of the symbols can be system specific. For example, the ProgramMemory symbols have alternate versions for systems with up to 4GB RAM and systems with more than 4GB or RAM. Additionally, if the IP address display does not look right, know that the IP addresses come in an array containing the address of each NIC on the IPC. It is possible that you simply need to change to the correct port.

In general, if something isn't working, take a look in the Server Symbol section of the TwinCAT HMI Configuration window to see if there are alternatives to the default mappings/bindings.



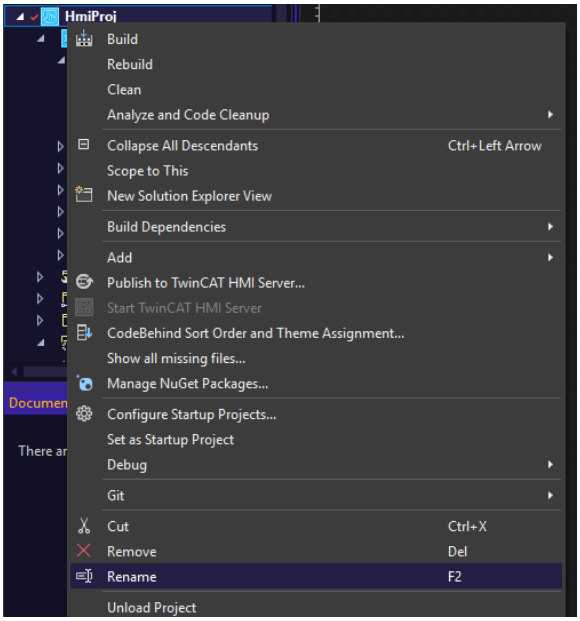
Project Renaming

The HMI project in the Template has been named as generically as possible. If a more application specific name is desired, steps to rename are outlined below.

Option 1 - Basic Rename

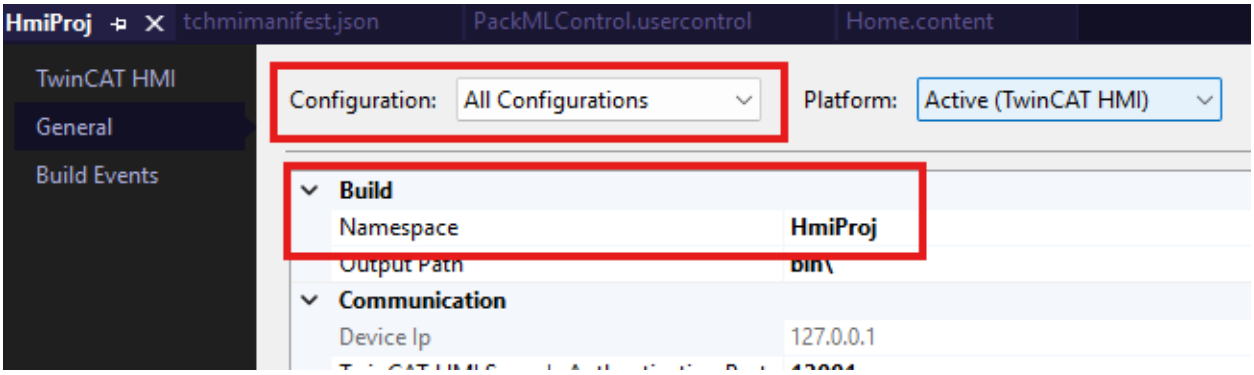
First, copy the appropriate project files to a new directory per the New Project Generation section of the Style Guide.

Open the solution and right-click on the HMI Project and rename.

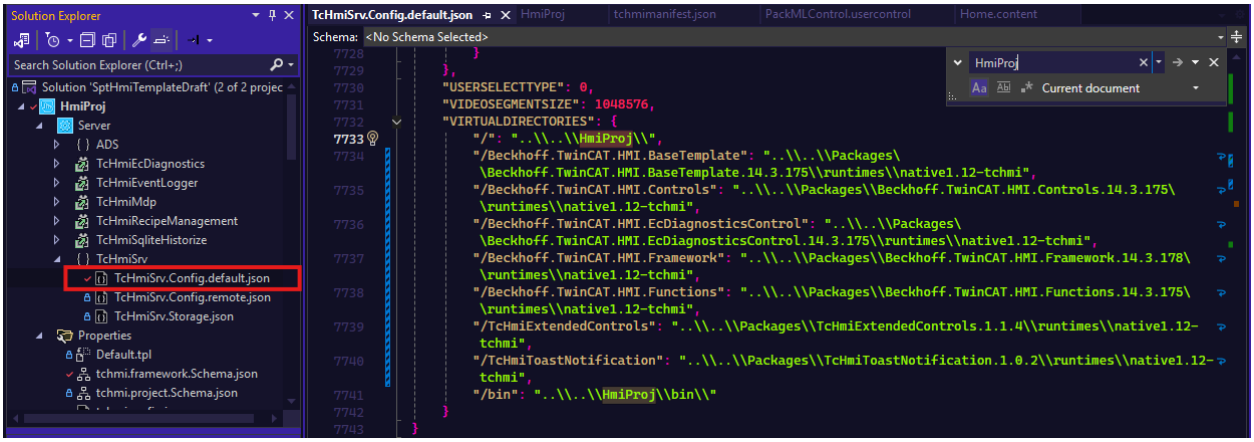


In Properties>tchmimanifest.json, change name and short_name.

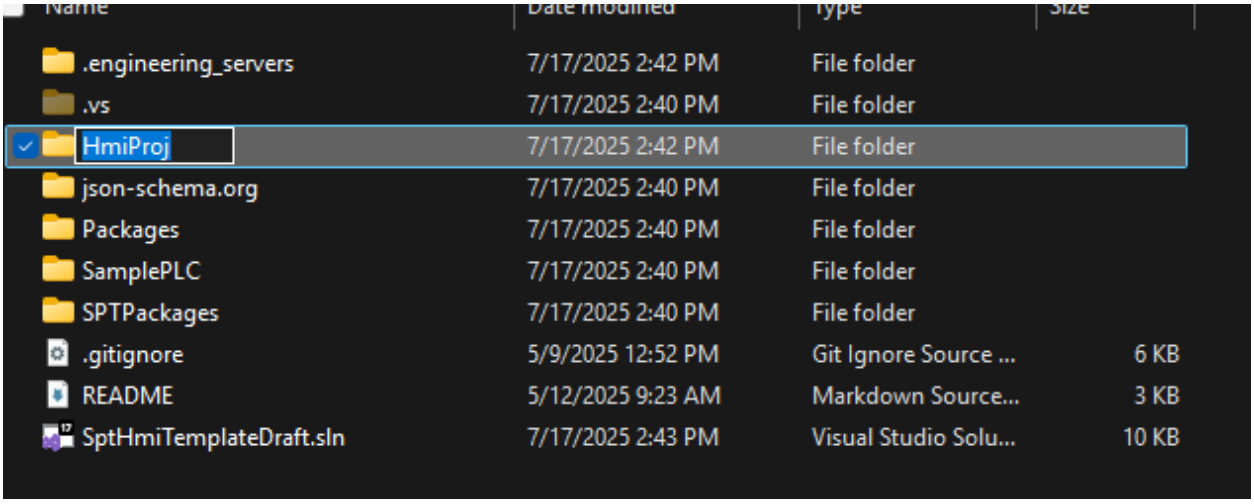
Right-click on the HMI Project and select Properties. Go to General, change Configuration to All Configurations and change Build>Namespace to your new name.



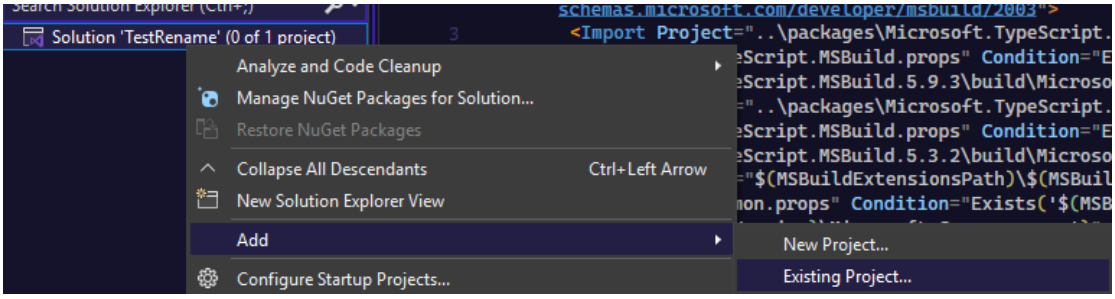
In TcHmiSrv.Config.default.json, change the file paths under VIRTUALDIRECTORIES from HmiProj to your new name.



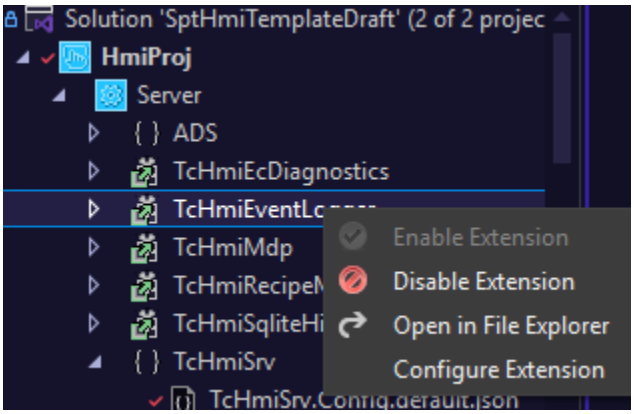
With the project closed, in the Project Directory, change the name of the folder containing the .hmiproj file.



When reopening the Solution, you will need to remove and readd the HMI Project via Add Existing.



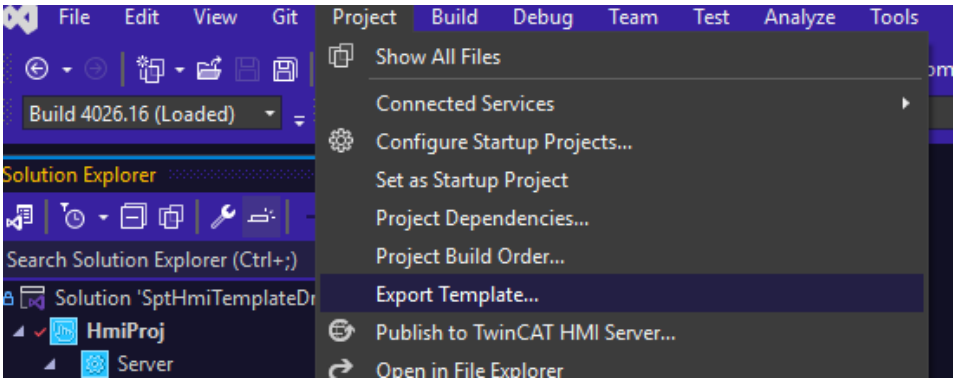
You may need to reenable server extensions, and any opened tabs may need to be closed and have the files reopened.



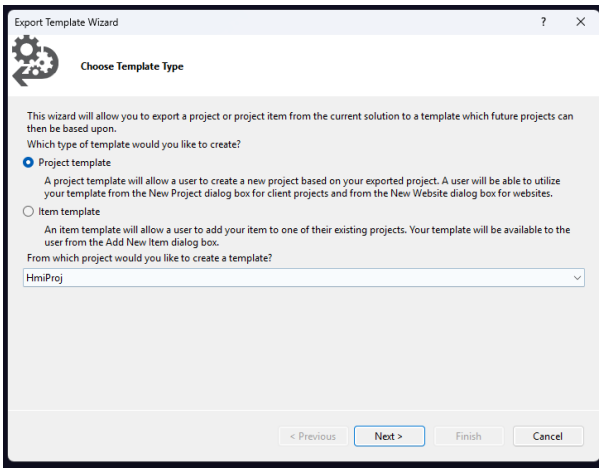
Option 2 - Create a Local Visual Studio Template

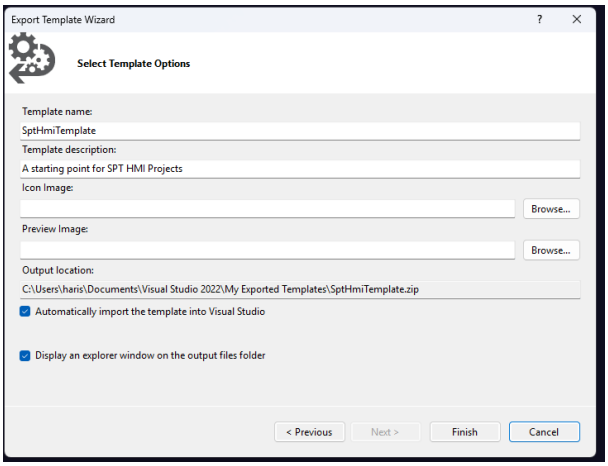
Open the cloned project.

Click on Project in the Menu bar and select Export Template.

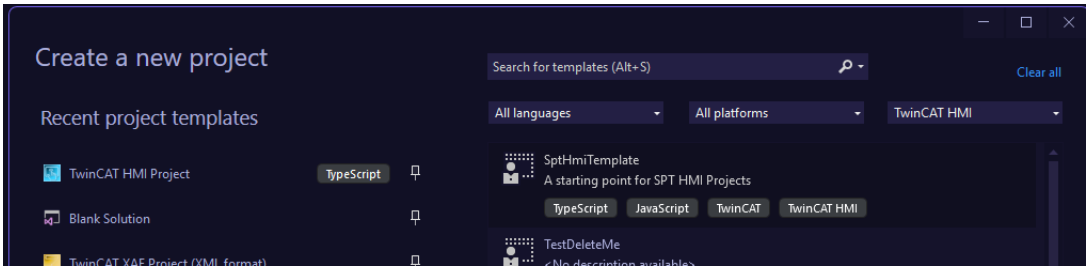


Follow the prompts and give the template a name like SptHmiTemplate.

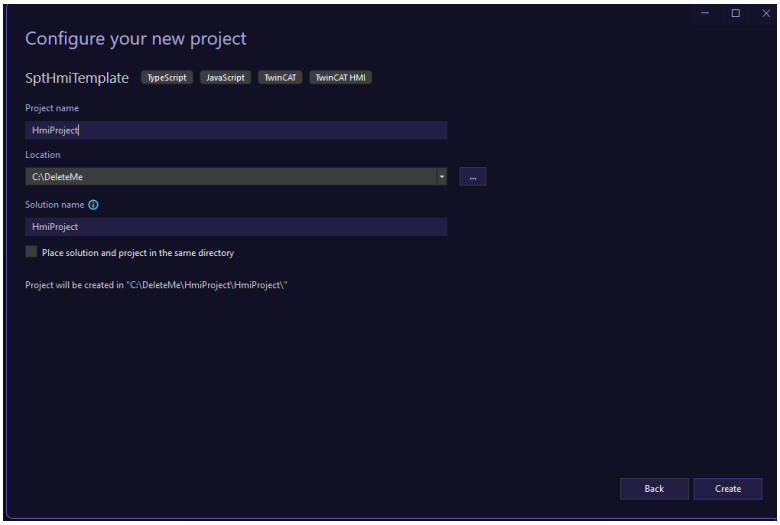




Once it is done, you can create a new project like you normally would, and can select the template.

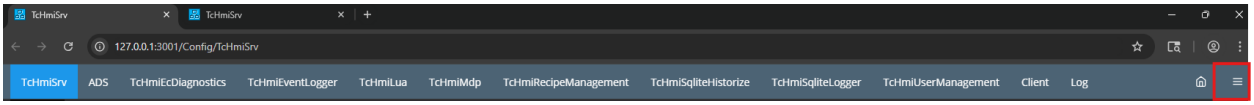


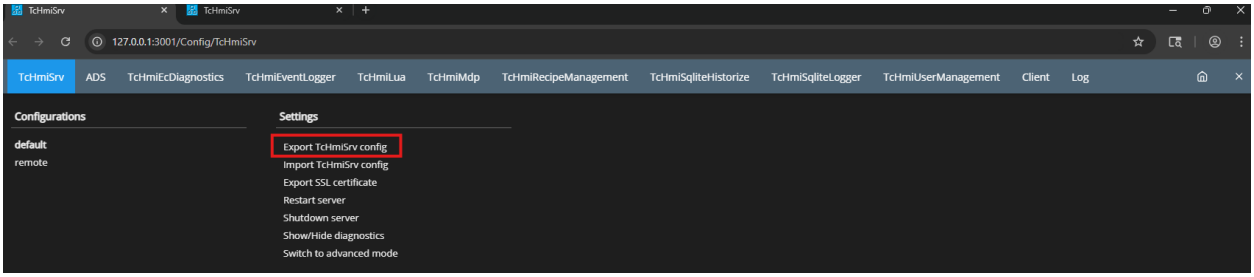
Giving it a specific name will then take care of all of the other places where the name is important.

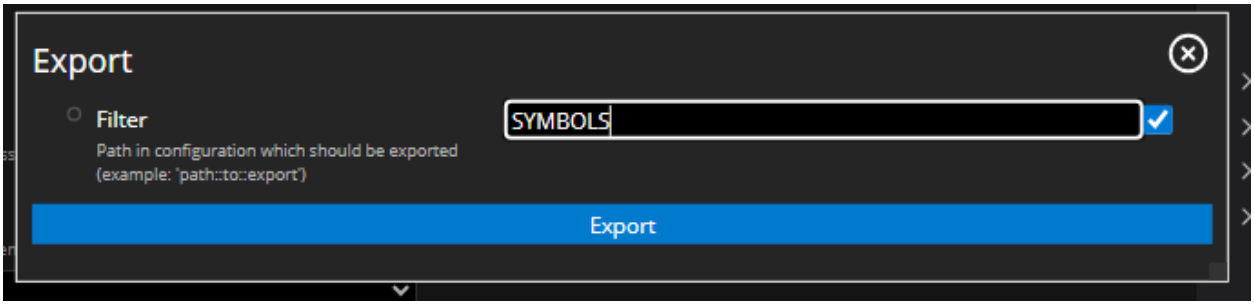


With both the cloned project and the new derived project open, you will need to port the TcHmiSrv configuration so that mapped symbols in the template come over.

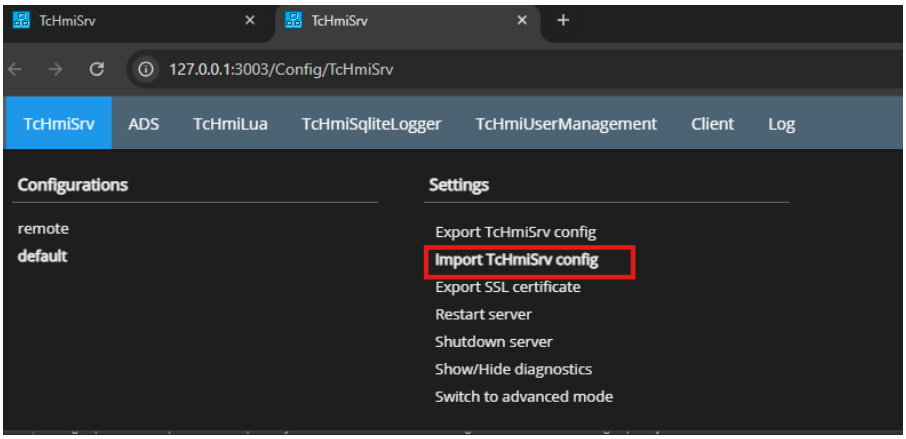
Go to the Config page of the cloned project via the server icon in the system tray, and export the TcHmiSrv (with a filter of SYMBOLS) config by navigating to the TcHmiSrv tab, clicking on the hamburger menu, and selecting Export.

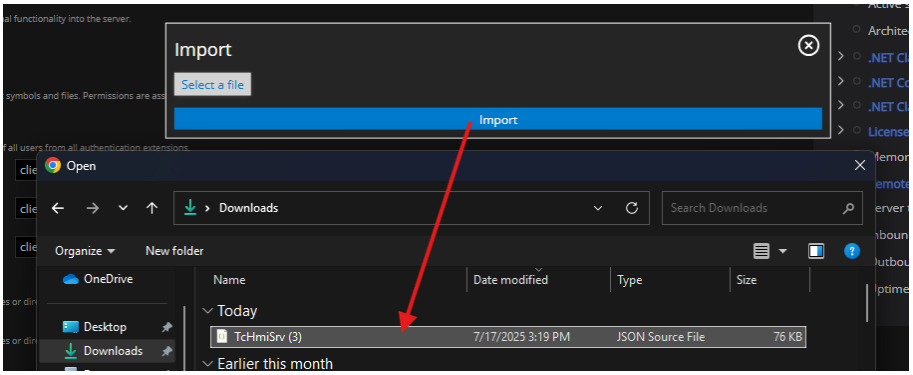






Finally, go to the config page of the new project and select Import in the hamburger menu on the TcHmiSrv tab.





Note - make sure to still update your local repo whenever starting a new project to ensure you have the latest changes and additions.

Font Usage Guidelines

Roboto Flex font family is used with an 1.8cqh font.

Roboto Flex upgrades Roboto so it becomes a more powerful typeface system. Roboto Flex is a variable font that you can customize to express and finesse your text in ways never before possible. Using a relatively new version of the OpenType file format, variable fonts allow one file to contain multiple stylistic variations, so the final file size of the complete package is remarkably small, given the range of expressive styles now available.

If additional font sizes are needed, four should be used at most. For example –

H1, H2, H3, Body

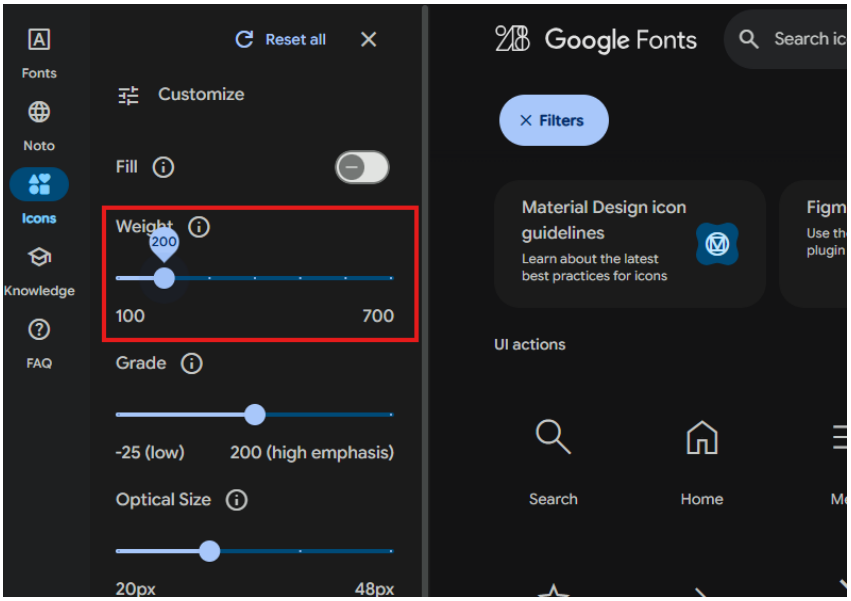
Usage Guidelines for more granular font control:

1. Use a consistent font family across all UI components.
2. Headings should be clear and distinct from body text.
3. Maintain ample line spacing for readability.
    a. Line height 120% of font-size recommended

Adding Icons

All icons should come from Google Material Font Icons and placed in the \Images\Icons folder.

Color should be greyscale and should not be altered. Font weight should be set to 200. Icons should be downloaded while the page is in dark mode, as our CSS rules convert them from dark to light, when appropriate.



Adding Additional Folders

New top level folders will likely not need to be added. However, it may be necessary to add sub-folders. For example, any sub navigation pages, or pages within regions that are not the main region, should be in a sub-folder of the Pages folder.

Additionally, if functions, user controls, or css files start to become numerous, it may make sense to group related ones in their respective folders.

Project Handoff

Prior to sending a project to someone else or pushing to a repo, ensure the latest versions of SPT NuGet Packages are in the SPTPackages folder. Copy .nupkg files from Packages folder (which is ignored by git).

Once a customer has the project, they must copy these files to C:\ProgramData\Beckhoff\NuGetPackages on their engineering PC prior to opening the HMI Project.